Manual de Usuario — Respaldo y Restauración (Fase 2)

Guía práctica para ejecutar cargas, respaldos y restauraciones del proyecto.

Requiere Docker Desktop corriendo y PowerShell. Todos los comandos se ejecutan

desde la carpeta `Fase2/`.

1. Levantar el entorno

```
docker compose up -d db
```

Crea el contenedor `fase2-pg` (Postgres 17) con `summarize_wal=on` (necesario

para respaldos incrementales) y dos volúmenes:

- `fase2_pgdata`: volumen Docker persistente para los datos de la BD.
- `./backups`: carpeta del host, para que los respaldos sobrevivan aunque se

elimine el contenedor o el volumen de datos.

2. Aplicar el esquema

```
docker cp sql\schema.sql fase2-pg:/schema.sql
docker exec fase2-pg psql -U postgres -d olimpiadas_fase2 -f /schema.sql
docker cp sql\stored_procedures.sql fase2-pg:/stored_procedures.sql
docker exec fase2-pg psql -U postgres -d olimpiadas_fase2 -f /stored_procedures.sql
```

3. Generar los subconjuntos de carga (una sola vez)

Requiere tener `../Fase1/etl-csvs` disponible (ya viene en el repo).

```
python carga\generar_subconjuntos.py
```

Genera `carga/subconjuntos/{por_anio,por_deporte,por_deportista}/cargaN_*`,

cada uno con **solo las filas nuevas** respecto al paso anterior. Incluye los

datos relacionados de las diez tablas: población, clubes y relaciones

atleta–club, además de las entidades olímpicas.

No ejecute `Fase1/etl/run_load.sh` para cargar Fase 2. Ese script usa otro contenedor y otra base. `cargar_paso.ps1` ya reproduce el paso que hacía falta: después de cada `COPY` ejecuta `sql/derivar_equipos.sql`, reconstruye los equipos sobre el estado acumulado y enlaza `participacion.equipo_id` antes de validar y respaldar.

4. Ciclo de una ronda (ejemplo: por deportista)

```
# Carga 1
.\Carga\cargar_paso.ps1 -CarpetaCSV .\Carga\subconjuntos\por_deportista\carga1_atenas2004
.\Backup\respaldo_full.ps1 -Destino por_deportista/full_carga1

# Carga 2
.\Carga\cargar_paso.ps1 -CarpetaCSV .\Carga\subconjuntos\por_deportista\carga2_pekin2008
.\Backup\respaldo_incremental.ps1 -Destino por_deportista/incr1_carga2 -ManifestBase
por_deportista/full_carga1

# Carga 3
.\Carga\cargar_paso.ps1 -CarpetaCSV .\Carga\subconjuntos\por_deportista\carga3_londres2012
.\Backup\respaldo_incremental.ps1 -Destino por_deportista/incr2_carga3 -ManifestBase
por_deportista/incr1_carga2
```

Cada script de respaldo imprime la duración y registra una línea en `logs/backups.csv` (`backup,tipo,base,inicio,fin,duracion_s`).

Validar después de cada carga

```
docker exec fase2-pg psql -U postgres -d olimpiadas_fase2 -c "SELECT count(*) FROM participaciones"
docker exec fase2-pg psql -U postgres -d olimpiadas_fase2 -c "SELECT * FROM resultados LIMIT 20"
```

(para rondas con muchas filas, usar `SELECT ... LIMIT 20` *en vez de* `SELECT`)

Para validar y dejar salida reproducible de **las diez tablas**, incluyendo fecha/hora, conteos, muestras de `SELECT *` y comprobación de filas huérfanas:

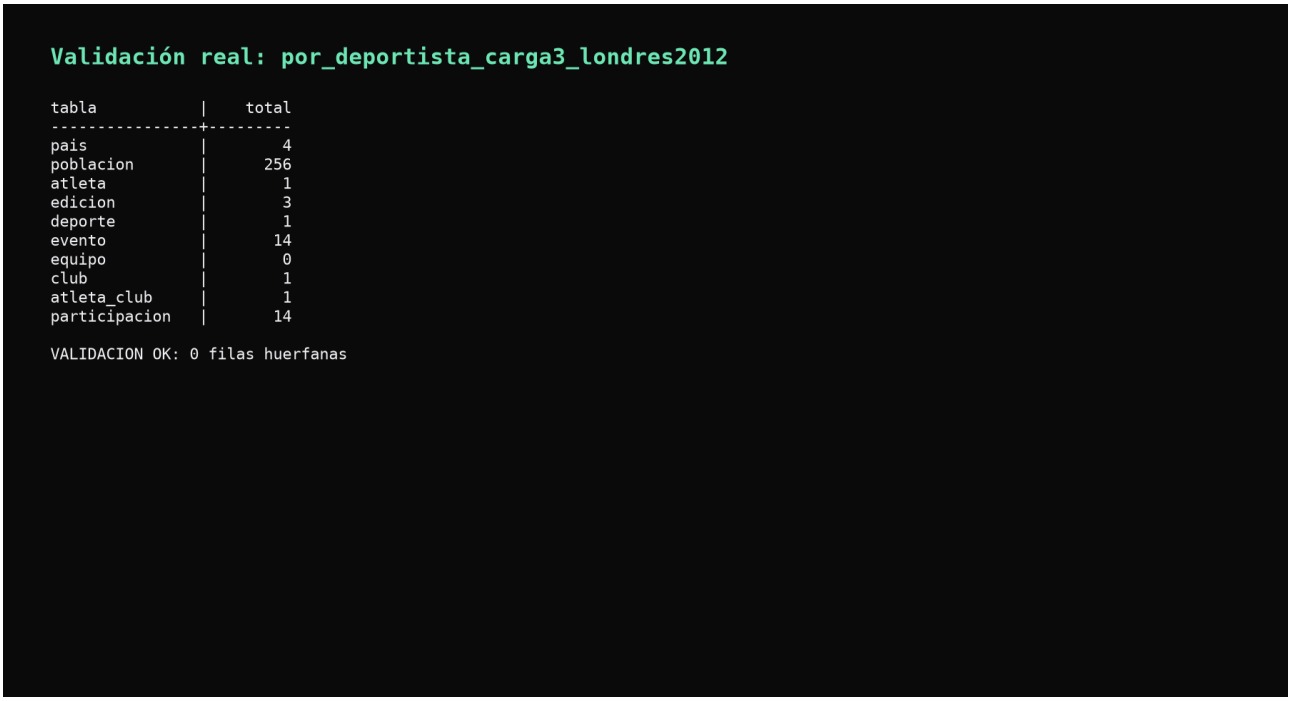
```
docker cp sql\validar_integridad.sql fase2-pg:/validar_integridad.sql
docker exec fase2-pg psql -U postgres -d olimpiadas_fase2 -f /validar_integridad.sql
```

Ejecute este bloque después de cada carga y después de cada restauración. Una ejecución correcta termina con `VALIDACION OK: 0 filas huerfanas`.

`cargar_paso.ps1` ejecuta esta validación automáticamente y guarda la salida

en `logs/cargas/<ronda>_<carga>.txt`. Puede desactivarse únicamente cuando sea necesario con `-Validar $false`.

Ejemplo de la validación generada automáticamente después de la tercera carga por deportista:



5. Eliminar la base de datos completa



Esto borra el contenedor **y** el volumen `fase2_pgdata`. Los respaldos en `./backups` no se ven afectados porque están en una carpeta del host.

6. Restaurar

6.1 Solo el respaldo completo



Copia el respaldo a `backups/restore_tmp`, levanta un contenedor Postgres apuntando ahí y mide el tiempo hasta que acepta conexiones. Al terminar solo tendrá los datos **hasta la carga 1** (el full backup no incluye lo

cargado después).

```
Restauración FULL verificada

pais            |      207
poblacion       |    8,832
atleta          |   11,100
edicion         |        1
deporte         |       50
evento          |      361
equipo          |    1,071
club            |    1,446
atleta_club     |    1,810
participacion   |   14,891

VALIDACION OK
```

6.2 Respaldo completo + incrementales (recuperación total)

```
./restore_restaurar_incremental.ps1 -Cadena
"con_deportista/full_carga1,con_deportista/incr1_carga2,con_deportista/incr2_carga3" -Puerto 5544
-NombreContenedorDestino "pasea-prueba"
```

Usa `pg_combinebackup` para reconstruir el directorio de datos completo a partir del full + la cadena de incrementales, y luego levanta el contenedor.

Al terminar tendrá **todos** los datos (carga 1 + 2 + 3).

```
Restauración incremental completa verificada

pais            |      228
poblacion       |    8,896
atleta          |   40,754
edicion         |        3
deporte         |       85
evento          |    1,676
equipo          |    5,108
club            |    9,384
atleta_club     |   15,202
participacion   |   75,533

VALIDACION OK
```

Validar la restauración

```
docker exec fase2-prueba psql -U postgres -d olimpiadas_fase2 -c "SELECT count(*) FROM
participacion;"
docker exec fase2-prueba psql -U postres -d olimpiadas_fase2 -c "SELECT * FROM participacion LIMIT
10;"
docker rm -f fase2-prueba
```

7. Verificar fragmentación de las tablas

```
docker cp sql/verificar_fragmentacion.sql fase2-pgs:/verificar_fragmentacion.sql
docker exec fase2-pgs psql -U postgres -d olimpiadas_fase2 -f /verificar_fragmentacion.sql
```

Muestra el % de tuplas muertas por tabla (`pg_stat_user_tables`) y el detalle de espacio libre de `participacion` (`pgstattuple`). La carga usa `COPY` , pero la derivación idempotente de equipos ejecuta `UPDATE` y `DELETE` ; por ello pueden aparecer tuplas muertas hasta que `autovacuum` o `VACUUM` las limpien.

```
Fragmentación medida después de derivar equipos

tabla           | vivas    | muertas  | estimación %
----------------+----------+----------+-------------
participacion   |   75,533 |   46,526 |       38.12
atleta          |   40,754 |        0 |        0.00
atleta_club     |   15,202 |        0 |        0.00
club            |    9,384 |        0 |        0.00
poblacion       |    8,896 |        0 |        0.00
equipo          |    5,108 |    3,367 |       39.73
evento          |    1,676 |        0 |        0.00
pais            |      228 |        0 |        0.00
deporte         |       85 |        0 |        0.00
edicion         |        3 |        0 |        0.00

pgstattuple(participacion):
tuplas muertas físicas = 0
espacio libre reutilizable = 35.85%
Integridad referencial = 0 filas huérfanas
```

8. Interpretar `logs/backups.csv` y `logs/restores.csv`

- `backups.csv` : una fila por respaldo tomado, con tipo (FULL/INCREMENTAL), el respaldo base del que depende (si aplica) y la duración en segundos.
- `restores.csv` : una fila por restauración, con el tipo (RESTORE_FULL /

RESTORE_INCREMENTAL) y la duración total (copiar/combinar + arrancar Postgres + esperar que acepte conexiones).

El análisis comparativo de estos tiempos está en `analisis_comparativo.md`.

9. Consultar con los stored procedures (reutilizados de Fase 1)

El `CALL` y el `SELECT` deben ir en la **misma sesión de psql**, porque el procedimiento deja el resultado en una tabla temporal (solo visible en la conexión que la creó). Dos `docker exec` separados fallan con `relation "resultado_sp_info_atleta" does not exist`.

```
docker exec fase2_pg psql -U postgres -d olimpiadas_fase2 -c "
CALL sp_info_atleta('Usain St. Leo Bolt');
SELECT * FROM resultado_sp_info_atleta ORDER BY anio;
"
```

10. Limpieza final

```
docker rm -f fase2_prueba 2>/dev/null
docker compose down -v
```

11. Generar los entregables PDF

```
python docs/generar_pdfs.py
```

Genera `docs/documentacion_tecnica.pdf` (informe, análisis y evidencia) y `docs/manual_usuario.pdf`, además de las capturas de validación generadas a partir de los logs reales. Requiere Chrome o Edge instalado.

12. Verificación previa a la entrega

```
./verificar_entrega.ps1
```

Comprueba archivos requeridos, PDFs, sintaxis, conteo de la base activa y que

los respaldos físicos no se incluyan accidentalmente en Git.